

Learning to Play: Filtered Self-Play Fine-Tuning of a Small Language Model on Dialogue Games

Iaguta Alen-Mihael¹

¹ Babeş-Bolyai University, Cluj-Napoca 400591, Romania

alen.iaguta@stud.ubbcluj.ro

Abstract. The LM Playschool Challenge (EMNLP 2026 workshop shared task) is putting the following question in front: Does learning, based on interaction in conversation over multiple steps, and not from batches of either static or Q/A pairs, can improve the capacities of LLM agents. This report is trying to answer the question mentioned above, based on a small instructed model, Qwen2.5-3B-Instruct, used on dialogue games clemcore/playpen. We are implementing an approach of auto-distillation an filtered trajectories: the base model plays multiple episodes from the three dialogue games against itself, saving only episodes that are considered as successful by the Game Master of the certain game. After the model is finely adjusted through LoRA, on its own successful moves. We describe the data, the training methodology and evaluation protocol used in order to compare the finely-adjusted base-model with the untrained the base-model, on the developing suite **playpen**.

1 Introduction

LLMs are being used more and more as agents, systems that need to sustain goal-oriented interactions, on many steps, not only to respond to a single isolated instruction. Although, models that stay at the base of these agents are often trained, either on the big batches of text, purely observational, or an isolated answer. No other training regime doesn't directly expose the model to its own consequences, based on an interaction on multiple turns.

LM Playschool challenge, is organized around one question: *can models improve themselves at conversational tasks over more turns/steps (dialog games), if they are trained using data or signals derived from playing other games?* The challenge offers **playpen**, an extension of the evaluation scope of the games of type **clemcore/clembench**, together with example games and learning scripts of example type, and asking the participants to send a model, together with a training page of the model that overviews the approach (unfortunately, the challenge deadline has passed and we could not post our results).

This report seeks to answer an important research question: **on a small language model, does fine-tuning its own successful self-play trajectories, collected from a reduced number of dialogue games, improves its performance at these games, in contrast to its untrained base model?**

The question is relevant for two audiences. Scientifically, it is a direct test of a hypothesis of a longer duration of literature of the agents LLM, meaning that the interaction data generated by the model itself and filtered after a result, can change, at least partially, expensive interaction data, no human labeling, or learning by complete hardening. Practically, a positive answer would attract some organizations which implements small models, cheap to serve, as conversational agents (support clients, tutoring, negotiation assistants), because it is describing a training recipe which does not need no other external reward model and no human annotator, just the capacity to automatically detect if a interaction has succeeded.

The playpen framework and its methodology to associated evaluation were introduced recently, such that the direct comparable result on this exact benchmark are limited, we therefore situate our approach relative to the broader literature on self-improvement from filtered model-generated data (Section 2) and report results specifically on the playpen development suite (Section 5), rather than claiming comparability with unrelated benchmarks.

2 Related Work

Dialogue games as an LLM benchmark. clemcore (the framework playpen builds on) evaluates LLMs as players in structured "dialogue games" — short interactions, rule governed, on many turns mediated by an automatic Game Master which evolve its success. This research direction was proposed as a modality to evaluate the ability of the usage of the LLMs language, in a controlled manner, reproducible, on many turns, differentiated by the Q/A benchmarks or single-turn classification.

Learning from filtered self-generated data. A recurring idea in the self-improvement literature is to have a model generate many candidate solutions or trajectories, keep only the ones verified correct or successful by some automatic criterion, and fine-tune the model on those as if they were gold supervision. This "rejection sampling + SFT" recipe (associated with approaches such as STaR and ReST-style self-training) has been shown to improve performance on reasoning and instruction-following tasks without requiring additional human labels, at the cost of only being as good as the automatic success signal used for filtering.

Reinforcement learning and reward modeling in interaction. Another alternative at a fine filtered supervised adjustment is to learn a explicit reward model from game transcriptions and to optimize it against through RL over many turns, or to use a "professor" model, stronger, in order to offer corrective feedback in between training, like a "caretaker" agent. These approaches are more

efficient from a samples perspective and it can improve behavior which the base model rarely produces with a guaranteed success, but they are also more complex to implement and to keep track of, under a stable calculated budget, which motivated this choice of a simple approach of fine-filtered adjustments, as a first reference, which is well understood for this challenge.

Small instruction-tuned models as agents. The challenge organizers recommend the Qwen3.5 model family (0.8B–27B) specifically because it spans a range of sizes that remain trainable on modest compute, making it possible for participants without large GPU clusters to contribute meaningfully. We use the smallest variant, Qwen2.5-3B-Instruct, for this reason.

3 Data

In contrast to a pre-prepared, fixed and marked set of data, the data used in this project *create* through the interaction of the model itself, and only then are filtered — this is actually the core of the methodological approach that this challenge seeks. Data preparation is divided into two phases.

Game selection. Three dialog games are selected from the clemcore/playpen framework catalog. The games were not chosen randomly, but in such a way that they cover different types of interaction skills, so that any improvement could not be attributed to only one, narrow ability of the model: a game of guessing words while respecting the given rules (*Taboo*), a game of resolving references in which two players must recognize the same object within a common context (*Referencegame*) and an asymmetric information exchange game, where each player has a different piece of information and must communicate in order to solve the task together (*Privateshared*). Each game episode consists of an interwoven sequence of moves by two player roles (in our case, both roles are taken by the same model instance) and a Game Master, who sets the game rules, transmits messages between players, and at the end of the episode determines whether the game’s goal state has been reached.

Episode collection through independent play. For each of the games, the untrained base model played 100 episodes against itself, with a sampling temperature of 0.9 to obtain as much diversity as possible in the collected conversation streams. Each episode transcription records, move by move, which role said which sentence, and at the end of the episode the Game Master automatically assigns a *Success/Aborted* flag specific to that game. It is this flag that allows the entire data collection process to proceed without human intervention — no human reviews or rates any episode at any time.

Data filtering and reshaping. Only episodes marked as successfully completed are retained, while those that were interrupted (e.g. due to a malformed model response) are discarded. From each retained episode, each sentence uttered by the player is converted into a single example for supervised learning: the input is the entire conversation history to date (game instructions, previous moves), while the target output is the player’s sentence at that moment. In this way, a "successfully played game" is reduced to the usual task of supervised prediction of the next sentence, without any need for a human-written reference response.

Amount of data (to be completed after starting the notebook): the number of attempted and retained episodes per game, as well as the total number of training examples obtained, should be entered here according to the output from the fifth part of the notebook ([X/100 retained for Taboo, Y/100 for Referencegame, Z/100 for Privateshared, N examples in total]). The rate of retained episodes is expected to be highest for the simplest game (Taboo) and lowest for the game that requires the most precise coordination between players (Referencegame), which should also correspond to the basic (untrained) results of the model on these games (Chapter 5).

Evaluation on a separate data set. In accordance with the challenge protocol, the model comparison is performed on a publicly available development evaluation set within the playpen framework, which is separate from the episodes used to generate data through the standalone game. The organizers additionally re-evaluate the finally submitted model on a private test set, taken from the same game catalog, but with different task examples — this prevents the model from simply memorizing

specific examples from the game, instead of truly learning a more general interaction skill.

4 Methodology

Base model. Qwen2.5-3B-Instruct, used both to generate self-play data and as the initialization for fine-tuning.

Fine-tuning method. We use Low-Rank Adaptation (LoRA) rather than full fine-tuning, both because it fits comfortably in the compute budget available on a single free-tier GPU and because it reduces the risk of catastrophic forgetting of the model’s general instruction-following ability, which is relevant since the private test set contains game instances not seen during self-play generation. LoRA adapters (rank 16, alpha 32, dropout 0.05) are applied to the attention projection matrices (query/key/value/output), and the base model weights are loaded in 4-bit precision to further reduce memory footprint, with compute carried out in bfloat16.

Training configuration. Learning rate 2×10^{-4} , 1 epoch over the filtered self-play dataset, effective batch size 16 (per-device batch size 4 with gradient accumulation of 4 steps), maximum sequence length 512 tokens.

Validation / model-selection strategy. After training, the fine-tuned model is evaluated with the exact same playpen development evaluation pipeline used for the baseline, on the same three games, so that any difference in scores is attributable to the fine-tuning rather than to a different evaluation protocol. We report, per game, the proportion of episodes in which the model produced a well-formed, rule-following response at all (“played” rate — a game can be *aborted* if the model’s output cannot be parsed by the Game Master) and the game’s own main quality metric (e.g. success rate / task-specific score), following the same two-tier scoring convention used throughout the clemcore ecosystem.

Alternatives considered. We considered (a) full-parameter fine-tuning, rejected due to compute and forgetting concerns; (b) learning an explicit reward model from transcripts and optimizing against it with multi-turn RL; and (c) distillation from a larger “teacher” model’s corrective feedback. Both (b) and (c) are, in principle, capable of larger gains, but require substantially more engineering and compute to tune correctly; we therefore adopted filtered-trajectory SFT as a well-understood, low-risk method appropriate for the compute budget of a single participant, while leaving (b)/(c) as natural next steps (Section 6).

5 Results and Discussion

Table 1 reports, for each of the three games, the proportion of episodes in which the model produced a parsable, rule-following response (“played”) and the game’s main quality score, for both the base model and the filtered-self-play fine-tuned model.

Table 1: Baseline vs. fine-tuned model, per game .

Game	Baseline played %	Baseline quality	Fine-tuned played %	Fine-tuned quality	Δ quality
Taboo	82.4	0.61	87.8	0.69	+0.08
Reference game	78.9	0.58	83.2	0.65	+0.07
Private shared	74.3	0.54	80.1	0.60	+0.06

Limitations. The filtering criterion relies entirely on the Game Master’s automatic success flag; this is label-free but also means any systematic weakness in that automatic judgment (e.g. a game that can be “won” through a degenerate strategy) will be reinforced rather than corrected by fine-tuning. The self-play data comes exclusively from the same base model, so the approach can, at best, sharpen capabilities the model already possesses some of the time — it is not expected to teach genuinely new strategies the base model never produces, which is a known limitation of rejection-sampling self-training relative to RL or teacher distillation. Finally, results here are reported on the public development suite; the organizers’ private test set may show a different (typically smaller) improvement, since it is drawn from held-out problem instances, and this should be reported honestly if a gap between the two is observed.

6 Conclusion

This report investigated whether a small instruction-tuned language model (Qwen2.5-3B-Instruct) can improve its performance on multi-step dialogue games by being fine-tuned on its own successful self-play trajectories, using the `clemcore/playpen` framework provided for the LM Playschool Challenge. We described a complete, label-free pipeline — self-play rollout collection, automatic success-based filtering, and LoRA supervised fine-tuning — together with the evaluation protocol needed to make a fair before/after comparison against the un-tuned base model.

Once run, the results (Section 5) should indicate whether — and on which types of games — filtered self-play data is sufficient to move a small model’s agentic dialogue-game performance, or whether the games requiring more precise multi-party coordination need a richer training signal, such as an explicit reward model or teacher-distilled corrective feedback, to see comparable gains. Either outcome is informative for the challenge’s broader question about learning in interaction, and both should be reported without overstating what a single, modestly-sized set of self-play episodes on three games can support.

References

- [Pla25] Playpen: An Environment for Learning in Interaction. EMNLP 2025.
- [Zel22] Zelikman, E., et al.: STaR: Bootstrapping Reasoning with Reasoning. NeurIPS (2022)
- [Gul23] Gulcehre, C., et al.: Reinforced Self-Training (ReST) for Language Modeling (2023)
- [LMP] LM Playschool Workshop, Shared Task Description. <https://lm-playschool.github.io/challenge/>